

Gradient Descent from Scratch — Full Worked Example

Gradient Descent, Step by Step: Finding β Iteratively

Same dataset as before: $(1, 2), (2, 4), (3, 5), (4, 7)$

Goal: Instead of solving the normal equation exactly, find β_0, β_1 by repeatedly nudging them downhill on the cost surface, and confirm it converges to the same answer we found before: $\beta_0 = 0.5, \beta_1 = 1.6$.

Step 1 — Recap: Data and Hypothesis

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 4 \\ 5 \\ 7 \end{bmatrix}, \quad \hat{y} = \beta_0 + \beta_1 x, \quad n = 4$$

Terms: hypothesis, feature vector, target vector, sample size.

Step 2 — Recap: Cost Function

$$J(\beta_0, \beta_1) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{2n} \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 x_i))^2$$

This is a smooth, bowl-shaped (convex) function of β_0, β_1 — it has exactly one minimum, with no other dips or bumps to get stuck in. That's what makes gradient descent reliable here.

Terms: cost function, convex function, global minimum.

Step 3 — Deriving the Gradient (Calculus)

We need $\frac{\partial J}{\partial \beta_0}$ and $\frac{\partial J}{\partial \beta_1}$ — how much J changes as each parameter changes. Together these form the **gradient** ∇J , a vector pointing in the direction of steepest *increase* of J (so we move *opposite* to it to decrease J).

Derivative with respect to β_0 . Using the chain rule on $J = \frac{1}{2n} \sum (y_i - \beta_0 - \beta_1 x_i)^2$:

$$\frac{\partial J}{\partial \beta_0} = \frac{1}{2n} \sum_{i=1}^n 2(y_i - \beta_0 - \beta_1 x_i) \cdot (-1) = -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

Derivative with respect to β_1 . Same chain rule, but now the inner derivative with respect to β_1 picks up a factor of x_i :

$$\frac{\partial J}{\partial \beta_1} = \frac{1}{2n} \sum_{i=1}^n 2(y_i - \beta_0 - \beta_1 x_i) \cdot (-x_i) = -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_i$$

Summary — the two gradient formulas we will reuse every iteration:

$$\frac{\partial J}{\partial \beta_0} = -\frac{1}{n} \sum (y_i - \hat{y}_i)$$

$$\frac{\partial J}{\partial \beta_1} = -\frac{1}{n} \sum (y_i - \hat{y}_i) x_i$$

Terms: gradient, partial derivative, chain rule, steepest descent direction.

Step 4 — The Update Rule

At each iteration (“epoch”), update both parameters simultaneously:

$$\beta_0 := \beta_0 - \alpha \frac{\partial J}{\partial \beta_0}, \quad \beta_1 := \beta_1 - \alpha \frac{\partial J}{\partial \beta_1}$$

where α (**alpha**) is the **learning rate** — a small positive number controlling step size. Too large and it overshoots/diverges; too small and convergence is very slow.

Choice for this example: $\alpha = 0.1$

Terms: update rule, learning rate, epoch/iteration, simultaneous update.

Step 5 — Initialization

Gradient descent needs a starting guess. We initialize both parameters at zero:

$$\beta_0^{(0)} = 0, \quad \beta_1^{(0)} = 0$$

(The superscript (0) denotes “before iteration 1”.)

Terms: initialization, starting point.

Step 6 — Iteration 1 (fully worked)

Current parameters: $\beta_0 = 0, \beta_1 = 0$

Step A — compute predictions $\hat{y}_i = \beta_0 + \beta_1 x_i$ for all four points (both terms are 0, so every prediction is 0):

$$\hat{y} = [0, 0, 0, 0]$$

Step B — compute errors $e_i = y_i - \hat{y}_i$:

$$e = [2 - 0, 4 - 0, 5 - 0, 7 - 0] = [2, 4, 5, 7]$$

Step C — compute the cost (just to track progress):

$$J = \frac{1}{2(4)}(2^2 + 4^2 + 5^2 + 7^2) = \frac{1}{8}(4 + 16 + 25 + 49) = \frac{94}{8} = 11.75$$

Step D — compute the gradient for β_0 :

$$\frac{\partial J}{\partial \beta_0} = -\frac{1}{4}(2 + 4 + 5 + 7) = -\frac{18}{4} = -4.5$$

Step E — compute the gradient for β_1 (each error times its x_i):

$$\frac{\partial J}{\partial \beta_1} = -\frac{1}{4}(2(1) + 4(2) + 5(3) + 7(4)) = -\frac{1}{4}(2 + 8 + 15 + 28) = -\frac{53}{4} = -13.25$$

Step F — apply the update rule ($\alpha = 0.1$):

$$\begin{aligned}\beta_0 &:= 0 - 0.1(-4.5) = 0 + 0.45 = 0.45 \\ \beta_1 &:= 0 - 0.1(-13.25) = 0 + 1.325 = 1.325\end{aligned}$$

After iteration 1: $\beta_0 = 0.45, \beta_1 = 1.325, J_{\text{before this step}} = 11.75$

Step 7 — Iteration 2 (fully worked)

Current parameters: $\beta_0 = 0.45$, $\beta_1 = 1.325$

Step A — predictions $\hat{y}_i = 0.45 + 1.325x_i$:

$$\hat{y}_1 = 0.45 + 1.325(1) = 1.775$$

$$\hat{y}_2 = 0.45 + 1.325(2) = 0.45 + 2.65 = 3.10$$

$$\hat{y}_3 = 0.45 + 1.325(3) = 0.45 + 3.975 = 4.425$$

$$\hat{y}_4 = 0.45 + 1.325(4) = 0.45 + 5.30 = 5.75$$

Step B — errors $e_i = y_i - \hat{y}_i$:

$$e_1 = 2 - 1.775 = 0.225$$

$$e_2 = 4 - 3.10 = 0.90$$

$$e_3 = 5 - 4.425 = 0.575$$

$$e_4 = 7 - 5.75 = 1.25$$

Step C — cost:

$$\begin{aligned} J &= \frac{1}{8}(0.225^2 + 0.90^2 + 0.575^2 + 1.25^2) = \frac{1}{8}(0.050625 + 0.81 + 0.330625 + 1.5625) \\ &= \frac{2.75375}{8} \approx 0.3442 \end{aligned}$$

Step D — gradient for β_0 :

$$\frac{\partial J}{\partial \beta_0} = -\frac{1}{4}(0.225 + 0.90 + 0.575 + 1.25) = -\frac{2.95}{4} = -0.7375$$

Step E — gradient for β_1 :

$$\begin{aligned} \frac{\partial J}{\partial \beta_1} &= -\frac{1}{4}(0.225(1) + 0.90(2) + 0.575(3) + 1.25(4)) \\ &= -\frac{1}{4}(0.225 + 1.80 + 1.725 + 5.00) = -\frac{8.75}{4} = -2.1875 \end{aligned}$$

Step F — update:

$$\begin{aligned}\beta_0 &:= 0.45 - 0.1(-0.7375) = 0.45 + 0.07375 = 0.52375 \\ \beta_1 &:= 1.325 - 0.1(-2.1875) = 1.325 + 0.21875 = 1.54375\end{aligned}$$

After iteration 2: $\beta_0 \approx 0.5238$, $\beta_1 \approx 1.5438$, $J_{\text{before this step}} \approx 0.3442$

Notice how much closer we already are to the exact answer (0.5, 1.6) — and how much the cost dropped (from 11.75 to 0.34).

Step 8 — Iteration 3 (fully worked)

Current parameters: $\beta_0 = 0.52375$, $\beta_1 = 1.54375$

Step A — predictions:

$$\begin{aligned}\hat{y}_1 &= 0.52375 + 1.54375(1) = 2.0675 \\ \hat{y}_2 &= 0.52375 + 1.54375(2) = 0.52375 + 3.0875 = 3.61125 \\ \hat{y}_3 &= 0.52375 + 1.54375(3) = 0.52375 + 4.63125 = 5.155 \\ \hat{y}_4 &= 0.52375 + 1.54375(4) = 0.52375 + 6.175 = 6.69875\end{aligned}$$

Step B — errors:

$$\begin{aligned}e_1 &= 2 - 2.0675 = -0.0675 \\ e_2 &= 4 - 3.61125 = 0.38875 \\ e_3 &= 5 - 5.155 = -0.155 \\ e_4 &= 7 - 6.69875 = 0.30125\end{aligned}$$

Step C — cost:

$$\begin{aligned}J &= \frac{1}{8}((-0.0675)^2 + (0.38875)^2 + (-0.155)^2 + (0.30125)^2) \\ &= \frac{1}{8}(0.00455625 + 0.15112 + 0.024025 + 0.09075) \approx \frac{0.27045}{8} \approx 0.0338\end{aligned}$$

Step D — gradient for β_0 :

$$\frac{\partial J}{\partial \beta_0} = -\frac{1}{4}(-0.0675 + 0.38875 - 0.155 + 0.30125) = -\frac{0.4675}{4} \approx -0.1169$$

Step E — gradient for β_1 :

$$\begin{aligned}\frac{\partial J}{\partial \beta_1} &= -\frac{1}{4}(-0.0675(1) + 0.38875(2) - 0.155(3) + 0.30125(4)) \\ &= -\frac{1}{4}(-0.0675 + 0.7775 - 0.465 + 1.205) \approx -\frac{1.45}{4} \approx -0.3625\end{aligned}$$

Step F — update:

$$\begin{aligned}\beta_0 &:= 0.52375 - 0.1(-0.1169) \approx 0.52375 + 0.01169 \approx 0.53544 \\ \beta_1 &:= 1.54375 - 0.1(-0.3625) \approx 1.54375 + 0.03625 \approx 1.58\end{aligned}$$

After iteration 3: $\beta_0 \approx 0.5354$, $\beta_1 \approx 1.58$, $J_{\text{before this step}} \approx 0.0338$

We're now very close: $\beta_0 \approx 0.535$ vs. exact 0.5, and $\beta_1 \approx 1.58$ vs. exact 1.6.

Step 9 — Convergence Over More Iterations

Repeating Steps A-F mechanically (same formulas, just plugging in the newest β_0, β_1 each time) for many more iterations, with $\alpha = 0.1$:

Iteration	β_0 (start of step)	β_1 (start of step)	Cost J	β_0 (after update)	β_1 (after update)
1	0.0000	0.0000	11.75000	0.4500	1.3250
2	0.4500	1.3250	0.34422	0.5238	1.5438
3	0.5238	1.5438	0.03381	0.5354	1.5800
4	0.5354	1.5800	0.02536	0.5369	1.5861
5	0.5369	1.5861	0.02512	0.5367	1.5873
10	0.5346	1.5882	0.02510	0.5341	1.5884
15	0.5321	1.5891	0.02509	0.5316	1.5893
20	0.5297	1.5899	0.02507	0.5293	1.5900
25	0.5276	1.5906	0.02506	0.5272	1.5908
30	0.5256	1.5913	0.02505	0.5252	1.5914

Both parameters keep **very slowly** creeping toward (0.5, 1.6), and the cost keeps inching down toward the true minimum $J = 0.025$ (matched via the normal equation). This slow final crawl is a known behavior of plain gradient descent with a fixed, modest learning rate — the closer you get to the minimum, the smaller the gradient (the “slope”) becomes, so each step shrinks too.

Terms: convergence, plateauing, fixed learning rate.

Step 10 — Comparing to the Exact (Normal Equation) Answer

Quantity	Normal equation (exact)	Gradient descent (after 30 iterations)
β_0	0.5	0.5256
β_1	1.6	1.5913
Cost J	0.025	0.02505

Gradient descent is **approaching** the exact solution but hasn't perfectly reached it after 30 steps — it would keep closing the gap with more iterations, a smaller learning rate near the end, or a decaying learning rate schedule. For this tiny, well-conditioned problem the normal equation remains the faster and exact choice; gradient descent's value shows up on much larger problems where inverting $X^T X$ directly is impractical.

Terms: exact solution, approximate solution, learning rate schedule.

Step 11 — Final Prediction Using the Gradient Descent Result

Using the gradient descent parameters after 30 iterations ($\beta_0 \approx 0.5256$, $\beta_1 \approx 1.5913$) for the same new input $x = 5$ used before:

$$\hat{y} = 0.5256 + 1.5913(5)$$

Compute the multiplication:

$$1.5913 \times 5 = 7.9565$$

Add the intercept:

$$\hat{y} = 0.5256 + 7.9565 \approx \boxed{8.482}$$

Compare to the **exact** prediction from the normal equation: $\hat{y} = 8.5$. Gradient descent gets close (8.48 vs. 8.5) but not exact after only 30 steps — with more iterations, it would converge all the way to 8.5.

Summary

Order	Concept	Result
3	Gradient formulas	$\partial J/\partial\beta_0 =$ $-\frac{1}{n}\sum e_i, \partial J/\partial\beta_1 =$ $-\frac{1}{n}\sum e_i x_i$
4	Update rule	$\beta := \beta - \alpha \partial J/\partial\beta$
5	Initialization	$\beta_0 = 0, \beta_1 = 0$
6	After iteration 1	$\beta_0 = 0.45, \beta_1 = 1.325$
7	After iteration 2	$\beta_0 \approx 0.524, \beta_1 \approx 1.544$
8	After iteration 3	$\beta_0 \approx 0.535, \beta_1 \approx 1.58$
9	After iteration 30	$\beta_0 \approx 0.526, \beta_1 \approx 1.591$
11	Prediction at $x = 5$ (approx.)	$\hat{y} \approx 8.48$ (exact: 8.5)